

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer & Information System Engineering

CS-222 — Database Management Systems

CinemaDB

Intelligent Movie Streaming & Recommendation Database System

COMPREHENSIVE EVALUATION PROJECT (CEP) REPORT

SUBMITTED BY:

Umaima Mashhood (CS-24055)

Rumaisah (CS-24063)

Batool Zehra (CS-24056)

Muhammad Usman (CS-24074)

SUBMITTED TO:

Miss Hameeza

GITHUB LINK: <https://github.com/usmxnshahzad/cinemadb>

May 2026

Table of Contents

- 1 Introduction 3
- 2 Problem Statement..... 3
- 3 System Requirements 3
 - 3.1 Functional Requirements 4
 - 3.2 Non-Functional Requirements..... 4
- 4 Entity-Relationship Model..... 4
 - 4.1 Entities and Attributes..... 4
 - 4.2 Relationships and Cardinalities..... 5
- 5 Normalization..... 5
 - 5.1 USERS & USER_DETAILS..... 5
 - 5.2 MOVIE Table..... 5
 - 5.3 MOOD_SESSION & CHAT_LOG 5
 - 5.4 Activity Tables 5
 - 5.5 Normalization Summary..... 6
- 6 Normalized Schema 6
- 7 DDL — Table Creation 7
- 8 Indexing Strategy..... 9
 - 8.1 B-Tree Indexes..... 9
 - 8.2 FULLTEXT Indexes..... 9
- 9 Application Architecture 10
 - 9.1 Backend — Flask..... 10
 - 9.2 Frontend — HTML/CSS/JS 10
 - 9.3 Recommendation Engine 10
- 10 Key SQL Queries..... 10
- 11 Key Features..... 11
- 12 UI Showcase 12
- 13 Conclusion..... 15

1. Introduction

CinemaDB (IMSR-DB) is a full-stack intelligent movie streaming and recommendation database system built for the CS-222 Database Management Systems course at NED University. The project goes beyond a simple catalogue — it is designed to mirror how production streaming platforms handle user data, content discovery, and personalized recommendations at the database level.

The backend is a Flask (Python) application backed by MySQL. Data for 200 top-rated films was sourced from a Kaggle IMDB CSV dataset and enriched with movie posters, trailers, and keyword metadata fetched from the TMDB API. The frontend uses a dark, cinema-inspired HTML/CSS/JavaScript interface. The recommendation engine is built on TF-IDF and cosine similarity, with a mood-based filtering layer on top.

2. Problem Statement

Streaming platforms are data-intensive by nature. They must handle large user bases, growing movie catalogues, continuous watch histories, personalized preferences, and real-time recommendation demands — all without introducing redundancy or sacrificing query performance. CinemaDB was designed to tackle these same challenges at an academic scale using proper relational database principles. The specific objectives were:

- Design a fully normalized relational schema (up to BCNF) covering all core streaming platform data
- Populate the database with 200 real films from an IMDB Kaggle dataset, enriched via the TMDB API
- Implement role-based access control distinguishing regular users from admins
- Support personalized features — watchlist, favourites, ratings with reviews, and watch history
- Build a content-based recommendation engine using TF-IDF and cosine similarity
- Add a mood-based Mood Finder quiz that maps emotional state and viewing context to genre weights
- Integrate a rule-based chatbot (CineBot) with full conversation history stored in the database
- Apply strategic B-tree and FULLTEXT indexing across all performance-critical query paths

3. System Requirements

3.1 Functional Requirements

From a user's perspective, the system supports:

- Registration, login, and profile management — username, avatar, bio, and hashed password
- Browsing the movie catalogue with filters for genre, rating, and sort order
- Full-text search across title, director, cast, and keyword columns
- Viewing a movie's detail page with embedded YouTube trailer, genres, cast, director, and synopsis

- Toggling movies in and out of a personal watchlist or favourites list
- Submitting a star rating (1–10) and an optional written review per movie
- Taking the Mood Finder — a 4-step quiz (feeling, company, intensity, runtime) that returns 12 matched films
- Chatting with CineBot for natural-language movie queries (rule-based, 20+ intents)
- Admin panel with live platform statistics, user management, ratings moderation, and chat log access

3.2 Non-Functional Requirements

- BCNF normalization across all 13 tables to eliminate redundancy
- B-tree and FULLTEXT indexes on all performance-critical columns
- Werkzeug password hashing for secure credential storage
- Foreign key constraints with deliberate ON DELETE CASCADE and ON DELETE SET NULL strategies
- Parameterized SQL queries throughout to prevent SQL injection
- Request-scoped MySQL connections using Flask's application context (g)

4. Entity-Relationship Model

4.1 Entities and Attributes

Nine core entities were identified after analyzing the system requirements. The movie content data is further decomposed into four satellite tables (movie_genre, movie_director, movie_cast, movie_keywords) to satisfy 1NF:

Entity	Key Attributes	Primary Key
users	id, username, email, password, role, avatar, bio, created	id
user_details	email, username, password, role, avatar, bio	email
movie	Movie_id, title, description, rating, year, runtime, Poster, Trailer	Movie_id
watch_history	id, user_id, movie_id, watched_at	id
watchlist	id, user_id, movie_id, added_at	id
ratings	id, user_id, movie_id, score, review, posted_at	id
favourites	id, user_id, movie_id, added_at	id
mood_session	id, user_id, answers_json, genre_map_json, created_at	id
chat_log	id, user_id, role, message, ts	id

4.2 Relationships and Cardinalities

The following relationships govern how the entities interact:

Relationship	Entities Involved	Cardinality	Description
Writes	users ↔ chat_log	1 : M	Each user can log many chat messages; each message belongs to one user
Creates	users ↔ mood_session	1 : M	A user can run many mood quiz sessions
Rates	users ↔ movie (via ratings)	M : N	Users rate many movies; movies are rated by many users. UPSERT on duplicate.
Watches	users ↔ movie (via watch_history)	M : N	Re-inserted on each view to keep most recent at top
Saves	users ↔ movie (via watchlist)	M : N	UNIQUE (user_id, movie_id) — no duplicate watchlist entries
Favourites	users ↔ movie (via favourites)	M : N	UNIQUE (user_id, movie_id) — toggle on/off via INSERT IGNORE / DELETE

5. Normalization

All tables were normalized systematically from 1NF through BCNF. The decisions below capture the key design choices made during the process.

5.1 USERS & USER_DETAILS

The original USERS table had a transitive dependency: email determined username, password, role, avatar, and bio — yet email was not the primary key. This violated 3NF. The table was split into two: users (id, email) and user_details (email, username, password, role, avatar, bio). The email column serves as the foreign key linking the two. Both resulting tables are in BCNF.

5.2 MOVIE Table

The original movie table stored genre, director, cast, and keywords as single columns — a direct 1NF violation since each could hold multiple values. The table was decomposed into five normalized relations: movie, movie_genre, movie_director, movie_cast, and movie_keywords. Each satellite table uses a composite primary key of (Movie_id, value). The avg_rating column was removed during 3NF analysis because it is a derived value computable from the ratings table — retaining it would introduce a transitive dependency.

5.3 MOOD_SESSION & CHAT_LOG

mood_session stores answers and genre scores as JSON/TEXT, making each cell atomic and satisfying 1NF. chat_log stores each message turn as a separate row with a role field ('user' or 'bot') and a timestamp — a deliberate structural choice that makes chat history queryable without deserialization. Both tables use ON DELETE SET NULL on the user foreign key so records survive account deletion as anonymous data.

5.4 Activity Tables

favourites, watchlist, watch_history, and ratings all have simple primary keys with no partial or transitive dependencies, placing them in BCNF by definition. favourites and watchlist enforce UNIQUE (user_id, movie_id) at the database level. ratings uses an ON DUPLICATE KEY UPDATE UPSERT so a user can revise their score without creating duplicate entries. watch_history omits the uniqueness constraint intentionally — the same film can be re-watched.

5.5 Normalization Summary

Table	1NF	2NF	3NF	BCNF	Notes
users	✓	✓	Split required	✓	email → user details resolved by splitting into user_details
user_details	✓	✓	✓	✓	email is PK; all attributes depend only on email
movie	Decomposed	✓	avg_rating removed	✓	Split into 5 tables; derived avg_rating removed
movie_genre	✓	✓	✓	✓	Composite PK (Movie_id, genre)
movie_director	✓	✓	✓	✓	Composite PK (Movie_id, director_name)
movie_cast	✓	✓	✓	✓	Composite PK (Movie_id, actor_name)
movie_keywords	✓	✓	✓	✓	Composite PK (Movie_id, keyword); max 8 per movie from TMDB
favourites	✓	✓	✓	✓	UNIQUE (user_id, movie_id); toggled via INSERT IGNORE / DELETE
watchlist	✓	✓	✓	✓	UNIQUE (user_id, movie_id)
watch_history	✓	✓	✓	✓	Duplicate entries allowed; re-inserted to refresh timestamp
ratings	✓	✓	✓	✓	UPSERT via ON DUPLICATE KEY UPDATE; score CHECK (1–10)
mood_session	✓ (JSON)	✓	✓	✓	answers_json and genre_map_json stored atomically as JSON/TEXT
chat_log	✓	✓	✓	✓	role is 'user' or 'bot'; each message is a separate row

6. Normalized Schema

After normalization, the database consists of 13 tables across three logical groups: user identity (users, user_details), movie content (movie + 4 satellite tables), and user activity (favourites, watchlist, watch_history, ratings, mood_session, chat_log).

Table	Primary Key	Foreign Keys	Key Constraints
users	id (AUTO_INCREMENT)	—	email UNIQUE NOT NULL
user_details	Email	email → users	role DEFAULT 'user'
movie	Movie_id (AUTO_INCREMENT)	—	rating DECIMAL(3,1)
movie_genre	(Movie_id, genre)	Movie_id → movie	CASCADE on delete
movie_director	(Movie_id, director_name)	Movie_id → movie	CASCADE on delete
movie_cast	(Movie_id, actor_name)	Movie_id → movie	CASCADE on delete
movie_keywords	(Movie_id, keyword)	Movie_id → movie	CASCADE on delete; max 8 per movie
favourites	id (AUTO_INCREMENT)	user_id, movie_id	UNIQUE (user_id, movie_id)
watchlist	id (AUTO_INCREMENT)	user_id, movie_id	UNIQUE (user_id, movie_id)
watch_history	id (AUTO_INCREMENT)	user_id, movie_id	No uniqueness — re-watch allowed
ratings	id (AUTO_INCREMENT)	user_id, movie_id	ON DUPLICATE KEY UPDATE; score CHECK (1–10)
mood_session	id (AUTO_INCREMENT)	user_id (SET NULL)	answers_json, genre_map_json as JSON
chat_log	id (AUTO_INCREMENT)	user_id (SET NULL)	role VARCHAR(10) — 'user' or 'bot'

7. DDL — Table Creation

The DDL below creates all 13 tables. ON DELETE CASCADE propagates deletes from parent to child for content-linked records. ON DELETE SET NULL is used for chat_log and mood_session so that removing a user preserves anonymous activity data in the database.

```
-- USERS (identity only; splits email from profile details)
CREATE TABLE users (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  email   VARCHAR(100) NOT NULL UNIQUE
);

-- USER_DETAILS (all profile data linked via email FK)
CREATE TABLE user_details (
  email    VARCHAR(100) PRIMARY KEY,
  username VARCHAR(50)  NOT NULL,
  password VARCHAR(255) NOT NULL,
  role     VARCHAR(20)  NOT NULL DEFAULT 'user',
  avatar   VARCHAR(255),
  bio      TEXT,
  FOREIGN KEY (email) REFERENCES users(email) ON DELETE CASCADE
);
```

```
-- MOVIE (core fields only; multi-value columns moved to satellite tables)
CREATE TABLE movie (
  Movie_id INT AUTO_INCREMENT PRIMARY KEY,
  title     VARCHAR(255) NOT NULL,
  description TEXT,
  rating    DECIMAL(3,1),
  year      INT,
```

```

        runtime      INT,
        Poster       VARCHAR(500),
        Trailer      VARCHAR(100)
    );

CREATE TABLE movie_genre      (Movie_id INT, genre      VARCHAR(50), PRIMARY KEY
(Movie_id, genre),            FOREIGN KEY (Movie_id) REFERENCES movie(Movie_id) ON DELETE
CASCADE);
CREATE TABLE movie_director (Movie_id INT, director_name VARCHAR(100), PRIMARY KEY
(Movie_id, director_name), FOREIGN KEY (Movie_id) REFERENCES movie(Movie_id) ON DELETE
CASCADE);
CREATE TABLE movie_cast      (Movie_id INT, actor_name   VARCHAR(100), PRIMARY KEY
(Movie_id, actor_name),      FOREIGN KEY (Movie_id) REFERENCES movie(Movie_id) ON DELETE
CASCADE);
CREATE TABLE movie_keywords (Movie_id INT, keyword      VARCHAR(100), PRIMARY KEY
(Movie_id, keyword),        FOREIGN KEY (Movie_id) REFERENCES movie(Movie_id) ON DELETE
CASCADE);

```

```

-- FAVOURITES & WATCHLIST (toggle tables, UNIQUE prevents duplicates)
CREATE TABLE favourites (
    id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, movie_id INT NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, movie_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (movie_id) REFERENCES movie(Movie_id) ON DELETE CASCADE
);

CREATE TABLE watchlist (
    id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, movie_id INT NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, movie_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (movie_id) REFERENCES movie(Movie_id) ON DELETE CASCADE
);

-- WATCH_HISTORY (no UNIQUE - same film can be re-watched; re-inserted to refresh
timestamp)
CREATE TABLE watch_history (
    id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, movie_id INT NOT NULL,
    Watched_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (movie_id) REFERENCES movie(Movie_id) ON DELETE CASCADE
);

-- RATINGS (UPSERT via ON DUPLICATE KEY UPDATE)
CREATE TABLE ratings (
    id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL, movie_id INT NOT NULL,
    score INT NOT NULL CHECK (score BETWEEN 1 AND 10),
    review TEXT, Posted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, movie_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (movie_id) REFERENCES movie(Movie_id) ON DELETE CASCADE
);

-- MOOD_SESSION (answers + genre scores stored as JSON)
CREATE TABLE mood_session (
    id INT AUTO_INCREMENT PRIMARY KEY, user_id INT,
    answers_json JSON NOT NULL, genre_map_json JSON NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL

```



```
);

-- CHAT_LOG (each message turn = one row; role is 'user' or 'bot')
CREATE TABLE chat_log (
  id INT AUTO_INCREMENT PRIMARY KEY, user_id INT,
  role VARCHAR(10) NOT NULL, message TEXT NOT NULL,
  ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);
```

8. Indexing Strategy

All indexes are created via ALTER TABLE ... ADD INDEX guarded by an information_schema.STATISTICS check — making them safe to run on every application startup and fully compatible with MySQL 5.6, 5.7, and 8.x (no IF NOT EXISTS syntax required). The check runs inside `_ensure_indexes_once()` in `app.py` and `ensure_indexes()` in `recommendation.py`.

8.1 B-Tree Indexes

Index Name	Table	Column(s)	Purpose
idx_movie_rating	movie	rating	Speeds up ORDER BY rating DESC on every trending/top-rated query
idx_movie_year	movie	year	Speeds up sort-by-year queries
idx_movie_title	movie	title	Fast prefix LIKE search on movie title
idx_movie_genre_mid	movie_genre	Movie_id	FK join in bulk load — avoids full table scan
idx_movie_genre_genre	movie_genre	genre	Genre-based filtering queries
idx_movie_director_mid	movie_director	Movie_id	FK join
idx_movie_director_name	movie_director	director_name	Fast director name search
idx_movie_cast_mid	movie_cast	Movie_id	FK join
idx_movie_cast_actor	movie_cast	actor_name	Fast actor name search
idx_movie_keywords_mid	movie_keywords	Movie_id	FK join
idx_movie_keywords_keyword	movie_keywords	keyword	Fast keyword search (TF-IDF engine)
idx_favourites_user_movie	favourites	user_id, movie_id	Composite — favourite toggle lookup
idx_watchlist_user_movie	watchlist	user_id, movie_id	Composite — watchlist toggle lookup
idx_watch_history_user	watch_history	user_id	Per-user history fetch
idx_watch_history_movie	watch_history	movie_id	Movie-side history queries
idx_watch_history_ts	watch_history	Watched_at	ORDER BY Watched_at DESC
idx_ratings_user_movie	ratings	user_id, movie_id	Composite — upsert and per-user rating fetch
idx_ratings_movie	ratings	movie_id	Per-movie rating aggregation
idx_ratings_posted	ratings	Posted_at	ORDER BY Posted_at DESC in admin /

Index Name	Table	Column(s)	Purpose
			movie page
idx_chatlog_user	chat_log	user_id	Per-user chat log fetch
idx_chatlog_ts	chat_log	ts	ORDER BY ts DESC in admin chat logs
idx_mood_user	mood_session	user_id	Per-user mood session fetch
idx_users_email	users	email	JOIN on users.email — used in nearly every auth query

8.2 FULLTEXT Indexes

FULLTEXT indexes are used for free-text search across description, director, cast, and keyword columns. These are also leveraged by the TF-IDF recommendation engine when building its feature strings.

Index Name	Table	Column	Purpose
ft_movie_description	movie	description	Full-text search on movie descriptions
ft_movie_director	movie_director	director_name	Full-text search on director names
ft_movie_cast	movie_cast	actor_name	Full-text search on cast names
ft_movie_keywords	movie_keywords	keyword	Full-text search on TMDB keywords (feeds TF-IDF engine)

```
ALTER TABLE `movie` ADD FULLTEXT INDEX `ft_movie_description` (description);
ALTER TABLE `movie_director` ADD FULLTEXT INDEX `ft_movie_director` (director_name);
ALTER TABLE `movie_cast` ADD FULLTEXT INDEX `ft_movie_cast` (actor_name);
ALTER TABLE `movie_keywords` ADD FULLTEXT INDEX `ft_movie_keywords` (keyword);
```

9. Application Architecture

9.1 Backend — Flask

app.py handles all routing, session management, and database operations. Routes are grouped by feature: authentication, movie CRUD, watchlist/favourites toggling, ratings, mood quiz, chatbot, and admin management. All queries use parameterized statements. MySQL connections are request-scoped via Flask's g object and torn down with @app.teardown_appcontext. The app seeds a demo admin account (admin@imsr.db / admin123) if no users exist on first startup.

9.2 Frontend — HTML/CSS/JS

The frontend renders the movie catalogue, detail pages with embedded YouTube trailers, user dashboards (watchlist, favourites, ratings, watch history), an admin panel, the Mood Finder quiz overlay, and the CineBot chat widget. JavaScript handles all AJAX calls — toggling watchlist/favourite status, submitting ratings, sending chatbot messages — without full page reloads.

9.3 Recommendation Engine

recommendation.py implements four techniques: (1) Content-Based Filtering using TF-IDF on a concatenated string of genre, keywords, and director — then cosine similarity between films; (2) Personalized scoring aggregating similarity across the user's watch history and favourites; (3) Mood-Based Mapping via a weighted MOOD_GENRE_MAP that converts quiz answers to genre scores; (4) a Hybrid Scoring blend of all three. Movies are loaded from MySQL into memory at startup, and a similarity matrix is pre-computed to serve instant recommendations.

10. Key SQL Queries

#	Function / Endpoint	Type	Description
Q5	login() POST	SELECT + JOIN	Authenticates user by joining users and user_details on email
Q6–8	signup() POST	INSERT × 3	Two-step signup: insert into users, then user_details; fetch back for session
Q10	movie_detail(mid)	DELETE + INSERT	Removes old watch_history entry and re-inserts with fresh timestamp
Q12	movie_detail(mid) reviews	SELECT + JOIN	Fetches latest 10 public reviews with usernames joined
Q13	admin() overview	SELECT × 5	Counts rows across favourites, ratings, watch_history, mood_session, chat_log
Q17	api_mood_recommend()	INSERT	Saves mood session (answers + genre map as JSON) to mood_session
Q18–20	api_favorite()	SELECT / DELETE / INSERT IGNORE	Three-step toggle: check → remove or add to favourites
Q24	api_rate()	INSERT ... ON DUPLICATE KEY UPDATE	UPSERT — inserts rating or updates existing score/review
Q25	api_chat()	INSERT × 2	Logs user message and bot reply as separate rows in chat_log
Q26	api_delete_user()	DELETE × 8	Admin cascade deletion across all user-related tables
Q30	api_admin_stats()	SELECT × 6	Live row counts from every major table for admin dashboard
Q32–36	_load_movies_from_db()	SELECT × 5	Loads all movies with genres, directors, cast, keywords for recommendation engine

11. Key Features

Authentication & Roles

Users register with email and a Werkzeug-hashed password. The role field in user_details distinguishes 'user' from 'admin'. Admin accounts access a live dashboard showing platform-wide statistics (users, films, favourites, ratings, watch events, mood sessions, chat messages) and content management controls. A demo admin account is seeded automatically on first startup.

Movie Catalogue & Search

200 real films sourced from a Kaggle IMDB CSV and enriched with TMDB API posters, trailers, and up to 8 keywords per film. Browsing supports genre/rating filters and sort order. Search is multi-column — querying title, director, cast, and keyword tables — backed by B-tree and FULLTEXT indexes for performance.

Watchlist, Favourites & History

All three use junction tables. Watchlist and favourites enforce UNIQUE (user_id, movie_id) at the database level via INSERT IGNORE and DELETE toggles. Watch history re-inserts (DELETE then INSERT) on each movie view, keeping the most recently watched film at the top without a uniqueness constraint.

Ratings & Reviews

Users submit a 1–10 score and optional review per movie. An ON DUPLICATE KEY UPDATE UPSERT allows revisions without duplicate rows. A CHECK constraint at the column level rejects out-of-range scores. Reviews are shown on the movie detail page with username and timestamp, fetched via a JOIN across ratings, users, and user_details.

Mood Finder

A 4-step interactive quiz asks: (1) How are you feeling right now?, (2) Who are you watching with?, (3) What intensity level suits you?, and (4) How much time do you have? Answers are fed into a weighted MOOD_GENRE_MAP that produces a normalized genre score dictionary. This map is stored in mood_session and used to re-rank the catalogue, returning 12 matched films. The quiz weights mood (1.0×), intensity (0.8×), and company (0.7×); runtime filters by length after scoring.

Recommendation Engine

recommendation.py builds a TF-IDF matrix at startup from a feature string per movie: genre + keywords + director. Cosine similarity is pre-computed across all 200 films. content_recommend() returns the top N similar films for any given movie. personalized_recommend() aggregates similarity scores across a user's entire favourites and watch history to surface a tailored picks list.

CineBot Chatbot

CineBot is a rule-based regex chatbot covering 20+ intents: film recommendations, genre queries, director lookups, top-rated lists, how-the-AI-works explanations, and general help. It does not use a language model. Every exchange — user message and bot reply — is logged as a separate row in chat_log with a role field and timestamp, making the full conversation history queryable from the admin panel.

13. UI Showcase

Landing Page

The home page opens with a full-width hero banner for the featured film (rating, genre, synopsis) and a Trending Now scroll row below. The top nav provides access to Browse, For You, Mood Finder, global search, and auth controls.

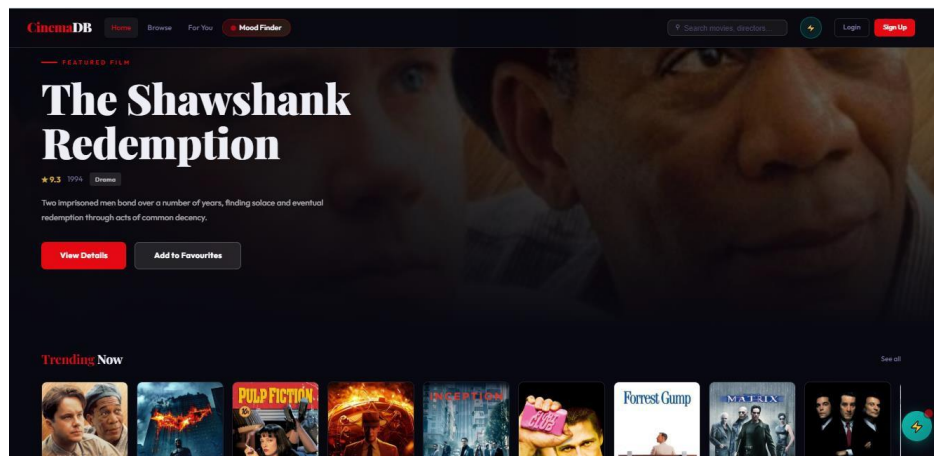


Fig 1. Home page — hero banner with Trending Now row

Login Page

The sign-in form sits centred on a dark background with the CinemaDB logo and tagline. A Sign Up link routes to the registration page. The same layout is used for sign-up.

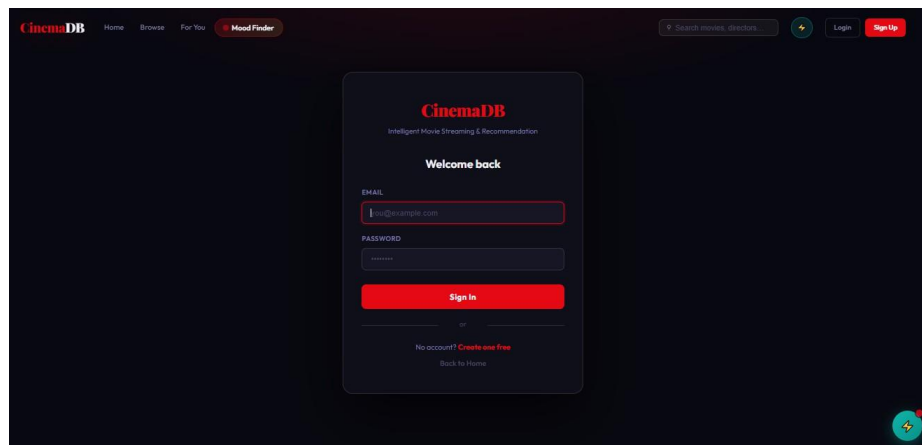


Fig 2. Login page

Browse & Search

The Browse page lists all 200 films with genre filter, rating filter, and sort controls. The result count is shown in red. Full-text search is available at the top of the page.

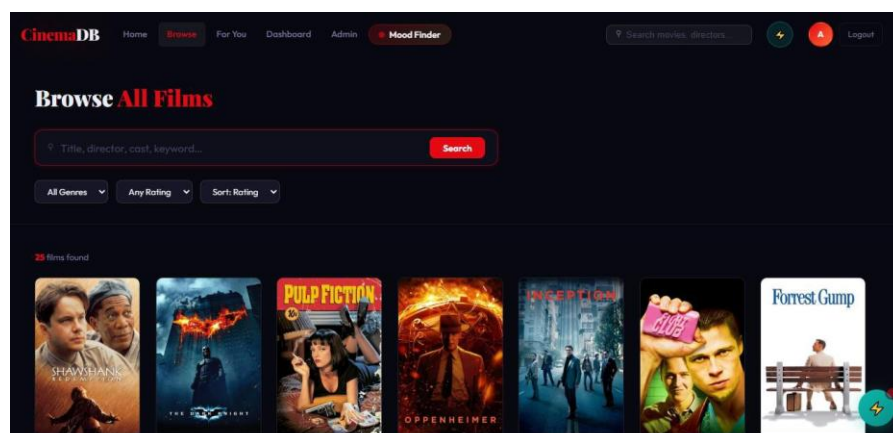


Fig 3. Browse page — filters, sort, and full film grid

Movie Detail Page

Each film's page shows the poster, rating badge, year, runtime, genre tags, director, cast chips, and synopsis. The Watch Trailer, Favourite, and Watchlist buttons sit below the metadata. A rating widget and public reviews appear further down the page.

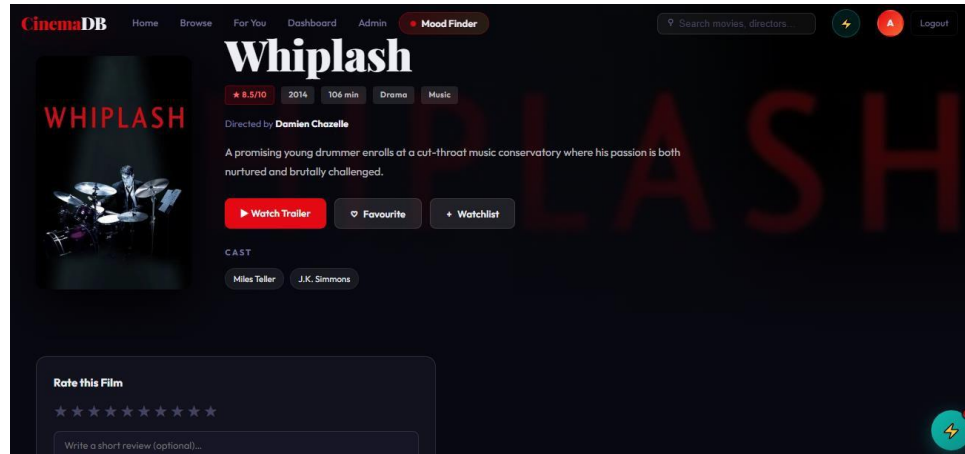


Fig 4. Movie detail — Whiplash with trailer, cast, rating interface

CineBot

CineBot is accessible from any page via the lightning bolt icon. The chat panel opens as a right-side overlay. The chatbot handles 20+ intents via regex pattern matching — genre queries, director info, top-rated requests, and how-it-works explanations. Every message is logged to the chat_log table.

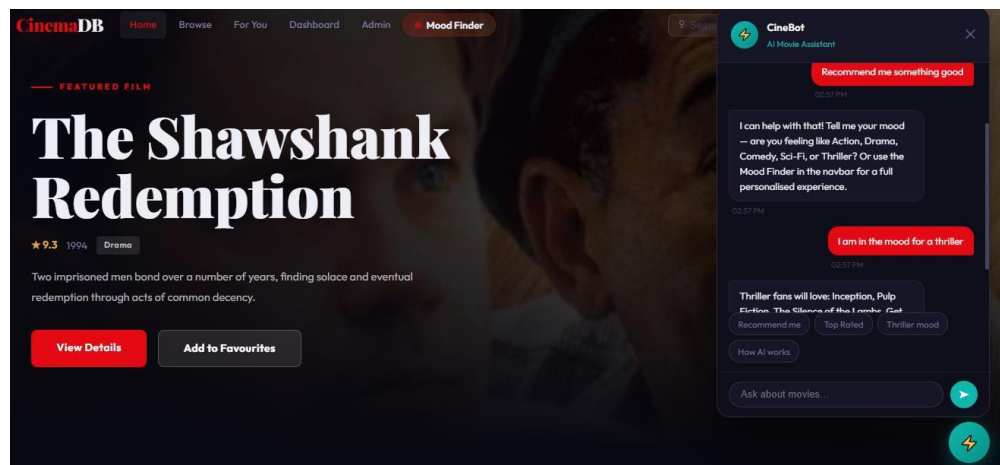


Fig 5. CineBot chatbot panel — natural-language movie queries

Admin Dashboard

Admin accounts see a live dashboard (auto-refreshes every 30 seconds) with platform-wide counters for users, films, favourites, ratings, watch events, mood sessions, and chat messages. The left sidebar navigates to user management, movie management, ratings, chat logs, and mood session tables.

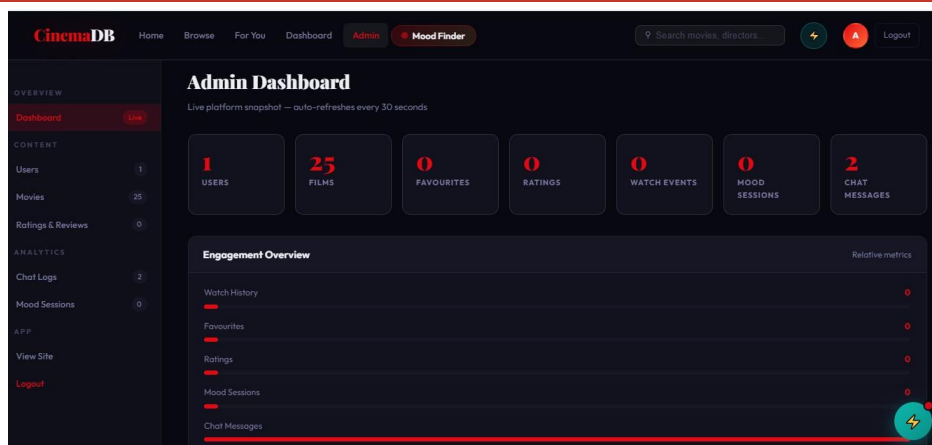


Fig 6. Admin dashboard — live stats and engagement overview

Mood Finder

The Mood Finder quiz runs across four steps, collecting the user's emotional state, viewing company, desired intensity, and available time. Each step presents visual choice cards. After submitting, the system computes a genre weight map and returns 12 matched films ranked by mood compatibility. The session is saved to mood_session in the database.

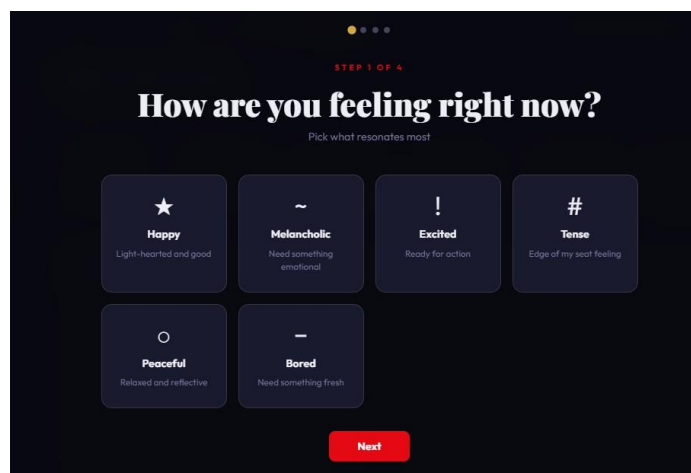


Fig 7. Mood Finder Step 1 — How are you feeling right now?

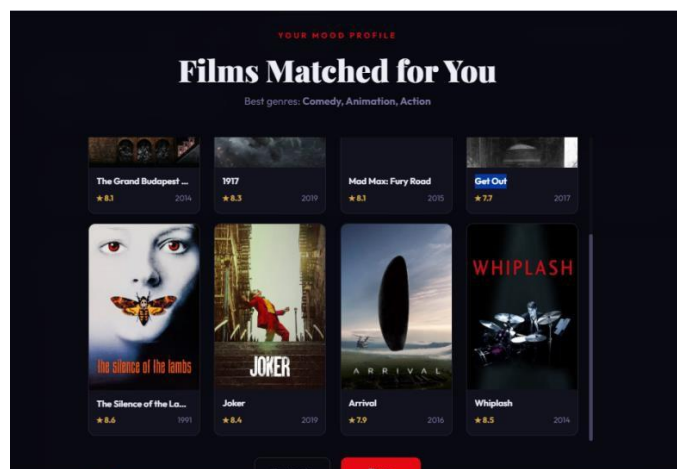


Fig 8. Mood Finder results — Films Matched for You

14. Conclusion

CinemaDB started as a schema design exercise and finished as a production-realistic streaming platform. The normalization process — decomposing the movie table into five satellite relations and splitting the user identity from profile data — produced a cleaner schema than the initial draft, and the discipline of working through each normal form had a direct, traceable impact on the final design. The project also validated that features typically associated with modern AI-powered applications chatbots, mood-based recommendations, and personalization engines integrate naturally into a relational model.

Overall, the project gave the team end-to-end experience with the complete DBMS development lifecycle: ER modelling, normalization, DDL, strategic indexing, query design, and full-stack application integration.